computers

MDPI

Article

Monitoring IoT and Robotics Data for Sustainable Agricultural Practices Using a New Edge–Fog–Cloud Architecture †

Mohamed El-Ouati, Sandro Bimonte * and Nicolas Tricot

TSCF, INRAE, University Clermont Auvergne, 63178 Aubiere, France; mohamed.el-ouati@inrae.fr (M.E.-O.)
* Correspondence: sandro.bimonte@inrae.fr

† This article is a revised and expanded version of a paper entitled Real-Time Monitoring and Active Control of Autonomous Agricultural Robot Trajectories Using an Edge-Fog Architecture, which was presented at Computational Science and Its Applications—(ICCSA) 2025 Workshops, Istanbul, Turkey, 30 June–3 July 2025.

Abstract

Modern agricultural operations generate high-volume and diverse data (historical and stream) from various sources, including IoT devices, robots, and drones. This paper presents a novel smart farming architecture specifically designed to efficiently manage and process this complex data landscape.The proposed architecture comprises five distinct, interconnected layers: The Source Layer, the Ingestion Layer, the Batch Layer, the Speed Layer, and the Governance Layer. The Source Layer serves as the unified entry point, accommodating structured, spatial, and image data from sensors, Drones, and ROS-equipped robots. The Ingestion Layer uses a hybrid fog/cloud architecture with Kafka for real-time streams and for batch processing of historical data. Data is then segregated for processing: The cloud-deployed Batch Layer employs a Hadoop cluster, Spark, Hive, and Drill for large-scale historical analysis, while the Speed Layer utilizes Geoflink and PostGIS for low-latency, real-time geovisualization. Finally, the Governance Layer guarantees data quality, lineage, and organization across all components using Open Metadata. This layered, hybrid approach provides a scalable and resilient framework capable of transforming raw agricultural data into timely, actionable insights, addressing the critical need for advanced data management in smart farming.

Keywords: data stream management systems; agricultural robots; anomaly detection; continuous queries; real-time data processing; edge processing

Check for updates

Academic Editor: Nagender Kumar Suryadevara

Received: 27 November 2025
Revised: 24 December 2025
Accepted: 29 December 2025
Published: 7 January 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Agricultural robots are autonomous machines designed for farm work. They are equipped with sensors to perceive their environment and actuators to carry out tasks, in order to operate independently of other robots and require no direct human intervention. These robots typically consist of a mobile base with a navigation system and a set of agricultural tools that can be mounted directly, semi-mounted, or towed. These robots are more and more frequently used in agro-ecology. Agro-ecology emphasizes environmentally sustainable farming practices. To support this, farmers and stakeholders need a robust system for supervising and remotely controlling agricultural robots [1]. Such a system should allow remote monitoring of robot progress in the field, as well as ingesting and analyzing data from various field devices [2], which can be classified as:

- *sensor data*, which are delivered in real-time from environmental sensors, e.g., temperature, wind speed, soil humidity;

- *odometry data*, which represent the robot's position, movement, and other mechanical parameters; such data are continuously updated, reflecting the current positions of robots in fields;
- *contextual data*, including spatial information (e.g., agro-field boundaries), and other farmer-specific details (e.g., the type of crops being harvested).

Sensors and odometry data are continuously generated as data streams, and they are accessed in real-time, requiring a monitoring system that can efficiently handle these dynamic streams [3]. For instance, it is crucial to track a robot's movement (for example speed, GPS location) and monitor weather conditions (for example, temperature, humidity), since farmers should be able to control the proper execution of agricultural tasks, monitor the robot's performance, and avoid any security, human, and animal risks. Moreover, agro-ecology needs historical data in order to analyze spatial and temporal patterns about crop health, growth, etc. In this context, also analysis of robots' behavior can provide interesting insight about mechanical faults, environmental blocking points, etc. Finally, such supervision systems must also provide remote control capabilities to allow farmers to control robots from their working position to change the robot's behavior when it deviates from the planned trajectory (e.g., robot has a delay) to guide it when the robot is not able to autonomously continue its work (e.g., the robot stops in the presence of an obstacle), etc.

Common end-users of such monitoring and supervision systems are agricultural stakeholders (e.g., farmers, agronomy decision-makers, and researchers). These users typically lack the advanced Information Technology (IT) skills required to implement their own queries over this data. Moreover, the involved data is highly complex, making it difficult to understand through simple tabular query results. Therefore, there is a clear need for ad-hoc visualization methods for real-time, historical, spatial, and multi-granular data.

In recent years, several works propose Big Data architectures to handle robotic and sensor data in the context of agriculture [4–7]. However, to the best of our knowledge, only [1] proposes an architecture (called *LambAgrIoT*) for robots monitoring and scheduling, based on a complex Big Data architecture (i.e., Lambda architecture). Although [1] presents an effective data management framework for the storage and analysis of IoT and robotic agricultural data, *LambAgrIoT* lacks in supporting (i) complex spatio-temporal analysis on stream data, (ii) effectively visualizing them, and (iii) managing, storing, and querying huge volumes of historical structured, unstructured, and semi-structured data.

To solve these challenges, in this paper, we extend the work of [8], which allows complex spatio-temporal analysis on stream data, by adding:

- A geovisualization approach for visualizing stream data [9]. *LambAgrIoT*'s data visualization remains relatively simple, limiting the effectiveness of end-users' situational awareness. Situational awareness refers to a person's ability to perceive and understand their environment, assess its elements, and anticipate possible changes [10]. In this context, we propose an improvement to the visual interface of *LambAgrIoT* by introducing geovisualization methods based on multi-granular spatial, temporal, and alphanumeric data representation [11]. Data are represented at different granularities, allowing for aggregated and detailed views when necessary. The proposed system thus allows (i) visualizing data from sensors and robots at different spatial (e.g., plot, GPS) and temporal (e.g., minute, second) scales; (ii) organizing and aggregating malfunction alerts based on priority, and (iii) remotely controlling the robots. The geovisualization approach developed covers all possible robot behaviors, from trouble-free agricultural operations to critical scenarios requiring autonomous decision-making by the robots or remote control by an operator.
- A Big Data architecture for storage and analysis of historical data. The foundational layer is built upon a distributed storage system. This system is tightly integrated

with a query and visualization platform that empowers end-users. The key features are: (i) simplified data access, end-users can query the stored data using familiar SQL commands; (ii) pre-defined reporting, the platform offers a suite of predefined visualization and reporting tools for immediate insights; and (iii) data discovery, due to the variety of collected and stored data, a centralized catalog system is essential. This system allows end-users to efficiently discover and locate the data they need using a simple keyword search interface.

Data Stream Management Systems (DSMSs) are used for real-time processing of scalable streaming data [12] across various domains, such as traffic and health, by utilizing continuous queries over defined time windows. However, applying DSMSs to agricultural robotics is difficult due to the high data rate, the need for real-time computation, and poor communication network quality in fields [13]. This poor quality causes data loss and leads to inaccurate monitoring indicators. A further challenge is the rigidity of DSMSs, which do not allow end-users to modify parameters such as query frequency or aggregation functions. To solve this, [8] extends *LambAgrIoT* with an edge-fog computing approach. At the edge (i.e., each robot), they deploy the Esper DSMS, enhanced with trajectory analysis capabilities via integration with H2GIS, to execute local queries and reduce latency. At the fog level, the spatial DSMS GeoFlink is used to compute and aggregate data and results from the robots. Crucially, the system features a dynamic time frequency mechanism that automatically adjusts the rate at which robots send data to the fog based on the robots' situation, using predefined rules. Furthermore, end-users are given the ability to remotely change the frequency of continuous queries to enable advanced control.

The paper is structured in the following way: Section 2 presents a theoretical framework for the design of the monitoring functionalities; Section 3 presents an overview of the proposed architecture; Sections 4–6 detail the different layers of the architectures: *Data source*, *Speed*, *Batch*, and *Ingestion* layers, respectively. Section 8 presents the related work, and Section 9 concludes the paper.

2. Monitoring Framework

Monitoring autonomous robots is a complex task that involves end-users (i.e., humans) and robots:

- Robots can have an autonomous behavior, or can be manually controlled by humans.
- Humans can provide actions or not when monitoring is not possible.

From our several years of real-life experiments, we propose a monitoring framework described in the next.

Figure 1 shows all different possible scenarios:

- Left-top quadrant: robot evolves in an autonomous way. The end user can visualize 'normal' data (e.g., position, speed, etc.), or check and handle the generated alerts (such as high speed, high engine temperature, etc.).
- Right-top quadrant: robot cannot evolve in an autonomous way (for example, when it is not able to avoid an obstacle), then the end-user manually and remotely controls it.
- Left-bottom quadrant: the end-user cannot receive any information from the robot (for example, when the communication network is down). Then, the robot cannot be controlled manually, but it must follow a particular behavior mode (for example, it stops its movement in order to avoid any important problems). Therefore, data is not sent in real-time, but it is sent off-line to the batch layer when communication is reestablished.

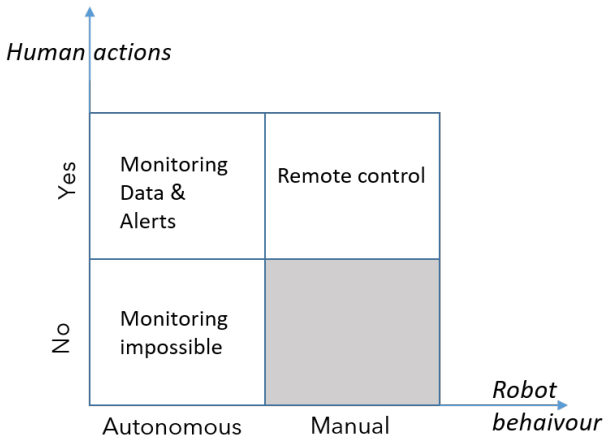

Figure 1. Monitoring framework.

This framework allows for the classification of different actions of end-users and robots, and therefore it can be used to define the different functionalities developed by the system and its architecture.

3. System Architecture's Overview

In this section, we present an overview of the smart farming architecture we propose extending and adapting the architecture proposed by [1], and refined in [8].

The proposed architecture (Figure 2) is structured into five distinct, interconnected layers, designed to efficiently handle the diverse and high-volume data (historical and stream) typical of modern agricultural operations: the *Source Layer*, the *Ingestion Layer*, the *Batch Layer*, the *Speed Layer*, and the *Governance Layer*. In particular, the layers are:

1. *Source Layer*: This layer is the entry point for all agricultural data. It groups the different devices and data storage systems that feed the architecture with all the necessary data. It accommodates various formats: structured alphanumeric and spatial data stored as JSON and CSV, which are then stored as tabular data in the relational DBMS PostGIS; images collected using drones; data from robots embedded with ROS, which can be streamed as ROS topics or saved as ROSbag files. These data are spatial, alphanumeric, and image data since they are collected by sensors equipped on robots; and stream numeric data from IoT devices. This layer has been proposed in [8].
2. *Ingestion Layer*: This layer manages data flow. A Python-based batch process handles bulk transfer of historical data. Simultaneously, Kafka serves as the central message broker, enabling high-throughput, fault-tolerant ingestion of real-time streams from robots and in-field sensors. This layer is deployed on the farm using a simple laptop machine, following the fog computing approach. This layer has been proposed in [1].
3. *Batch Layer*: The *Batch Layer* is dedicated to large-scale, historical data analysis. Storage is provided by a scalable Hadoop cluster. Querying and processing this historical data is supported by big data frameworks such as Spark, Hive, and Drill. The results are made available for visualization via Superset. This layer is deployed in the cloud since it requires significant storage and computational capabilities and does not require real-time responses. This layer is new according to [1,8]. In the original proposal of [1], the *Batch Layer* was implemented as a relational data warehouse, which presented some main limitations: (i) it does not scale, and (ii) it does not support complex data. Therefore, in this paper, we adopt a data lake approach, which is more flexible and provides better performance.

4. *Speed Layer*: This layer, proposed in [8], focuses on low-latency and real-time processing. Streaming data is processed by Geoflink (a spatial DSMS), which is coupled with a dedicated PostGIS database. Moreover, in this paper we provide a new ad-hoc geovisualization interface for real-time data analysis. This layer has been extended from [8].
5. *Governance Layer*: This layer allows for data organization, lineage tracking, and cataloging. It utilizes Open Metadata. This layer ensures that technical, business, and operational metadata are consistently managed across all components, from data acquisition to final visualization. This layer is new according to [1,8].

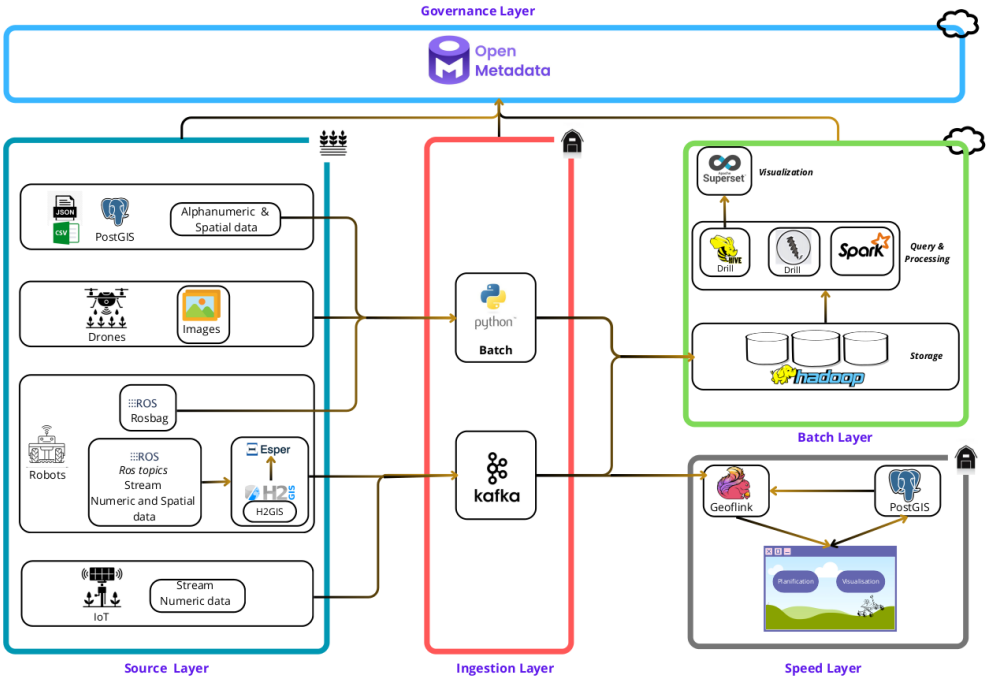

Figure 2. Edge–Fog–Cloud Architecture for monitoring IoT and robotics data for sustainable agriculture.

4. Data Source Layer

In this section, we describe the main data sources' devices and software we use (Section 4.1), and then we describe the computation at the edge for robotic data (Section 4.2).

4.1. Data Sources

Several data sources are used by smart farming applications, and therefore supported by our system.

Firstly, contextual data, such as plot descriptions, robot information, and agricultural practices information, are commonly stored in relational databases. In our application, we use PostGIS, which is an extension of PostgreSQL for spatial data, where farmers store transactional data about their robots and agricultural tasks. CSV and Excel files are also used by farmers to store information about crops, such as the year and the plot where each crop is present.

Secondly, we support data collected by sensors. Some of them store their acquisition in simple formatted CSV files, while others are able to communicate them in real-time. Data collected in real time by sensors, particularly those related to soil moisture, air humidity, soil temperature, and air temperature—represent key indicators for agro-ecology. Monitoring these variables helps to better understand interactions between plants, soil, and climatic conditions, thereby supporting more precise and sustainable management of natural resources. These data are transmitted in real-time to an Message Queuing

Telemetry Transport (MQTT) server, a lightweight publish–subscribe messaging protocol widely used for communication between devices and applications in IoT or machine-to-machine environments.

Thirdly, robot data are collected in real-time through the Robot Operating System (ROS). ROS is a flexible framework for writing robot software, providing tools, libraries, and conventions that simplify the task of creating complex and robust robot behavior across diverse hardware platforms. Although not a traditional operating system, ROS facilitates communication, code reuse, and distributed computing among multiple processes (nodes) within a robotics application. Each ROS node is a module responsible for a specific task, such as reading GPS coordinates, measuring speed, or monitoring the robot's status. These nodes publish their information on ROS topics, which serve as communication channels between different system components. For instance, a node publishes the robot's GPS position (latitude and longitude) on the '/robot/gps' topic, and another may publish speed data on '/robot/speed'. We also integrated images. These images provide visual representations of temperature variations across the study area. These images are typically captured using drones equipped with specialized infrared cameras that detect the infrared radiation emitted by objects and convert it into visible images, thereby enabling detailed spatial analysis of thermal patterns within the agricultural environment.

4.2. Edge Computing for Robots

Ref. [8] extends [1] with the capabilities of edge processing. In the proposal of [8], edge devices are considered as processing units, in addition to their function as data producers. Therefore, in [8] we designed a software stack for the edge devices (i.e., robots) in order to equip them with query processing capabilities. Processing streaming queries at the edge level requires DSMS that are lightweight. Indeed, robots come with a common laptop machine that has no high-performance physical devices such as RAM, CPU, and disks. Moreover, maintaining the software for robot fleets is a tedious and time-consuming task when the number of robots is high, or this task is left to the farmers. Therefore, the usage of complex DSMS, such as a distributed one, is not suitable for deploying DSMS at the robot level. This criterion along with the active maintenance of the DSMS software, and the need for the usage only of open-source software in our architecture lead to use Esper. A comparison of existing DSMS is provided in [8]. From this analysis, Esper appears to be the most suitable choice because it is lightweight and can be deployed on robots. Furthermore, it has advanced pattern-recognition capabilities, which are necessary for processing complex data (such as robot data), and it is actively maintained and updated. However, Esper does not support spatial data processing. Therefore, in [8] we extended Esper by creating a new trajectory point data type and an associated set of methods. These methods implement spatial operators by means of the native functionalities offered by H2GIS. H2GIS is an in-memory database supporting the storage and querying of spatial data. It provides better performance for data access than classical disk databases [14], making it compliant with the real-time performance required by robot monitoring queries. Moreover, it is a lightweight system, and consequently its maintenance is also easier. These spatial operators are then used in the continuous query methods offered by Esper to set frequency and window.

Further, our robots are equipped with ROS (Robotic Operating System). ROS provides a software framework for robot control and management. ROS uses a message subscribe protocol, which we use as a message queue to handle messages from the edge. Other light message broker systems could also be deployed. However, ROS allows the development of ad-hoc functionality; it does not include DSMS functionality. Thus, the usage of Esper plus HGIS allows for easy deployment of continuous queries on robots without any additional

ROS implementation. Moreover, this loosely coupled solution with Esper plus HGIS on top of the robot software allows for exploiting also other robots that do not use ROS, which makes our proposal more generic and flexible.

5. Speed Layer

The original *LambdAgrIoT* architecture uses Apache Sedona, which is a DSMS supporting spatial data within a distributed architecture. However, a more detailed analysis is needed to establish the best system being used. For that reason, we have compared GeoFlink, GeoMesa, and Apache Sedona according to different criteria as detailed in [8].This analysis reveals that *GeoFlink* seems to be the most suitable solution, as it excels in real-time processing, which is a priority for detecting anomalies in robot trajectories. Additionally, it offers good scalability and spatial support capabilities, while requiring only moderate resources. Therefore, we adopted GeoFlink in our new *Speed layer* implementation.

In this paper, we present a new geovisualization approach for smart farming data. Our geovisualization system relies on multi-granularity data modeling, allowing for the analysis and interpretation of robot information across different spatial, temporal, and thematic scales. The visualized data includes the robot's odometry data (i.e., speed, traction/adherence, etc.), robotic alerts (i.e., engine failure, zero speed, etc.), weather sensor data, and details of various agricultural practices (e.g., day, robot, equipment for plowing, spreading, etc.). For the spatial scale, data are represented at the agricultural plot scale and at the scale of each GPS point. The temporal scale includes different levels ranging from milliseconds to minutes. Two levels of robotic alerts have been defined: "alert" and "warning". They are represented in both an aggregated and a detailed manner. The description of the alerts takes into account the different types of users of the system (i.e., farmers, researchers, or external users) by offering personalized information for each user typology. The web interface, developed in JavaScript, consists of two main panels. On the left, the menu, and on the right, the following tools:

- *Monitoring* displays the movement of the robots via their GPS data and critical parameters, like speed and status, on an interactive map, viewable in 2D or 3D. Information is represented by colored points or gradient lines.
- *Diagnosis* lists the alerts by title, code, source, duration, and priority. Primary alerts are in red and secondary alerts are in yellow, visible until validated. They are sorted by priority, then by duration.
- *Scheduled Missions* displays the list of planned work with information stored in the DBMS.
- *Control* allows for piloting a robot via a joystick.
- *Weather* provides local weather data.
- *Planning* allows for defining and scheduling agricultural operations.

These different displays are synchronized and adapt to changes in data granularity. Figure 3a shows an example of the aggregated geovisualization of different alerts per plot. Each plot presents a colored icon (circle) that varies based on the number of alerts or warnings generated by the robots on the plot (i.e., orange for warnings and red for alerts). The user can click on a plot to visualize the specific data of a robot by viewing its exact trajectory (transition from an minute-by-minute GPS representation to a second-by-second representation), as well as the various alerts (red dot on the map accompanied by its detailed description) (Figure 3b). Moreover, our user interface allows remote control of robots movements by means of a simple joystick as shown in Figure 4. A video demonstrating all the functionalities of the interface is available here: https://youtu.be/zjCRQ9wFdoM (accessed on 26 December 2025).

Figure 3. Speed layer: Geovisualization of alerts: (**a**) plot—minute granularity, (**b**) GPS—10 ms granularity.

Figure 4. Speed layer: Remote control tool.

6. Batch Layer

The batch layer of the *LambdAgrIoT* architecture is dedicated to long-term storage and historical data analysis. The system must handle diverse sources and heterogeneous data structures, such as CSV and JSON files, databases, and images. Data generated by robots and IoT sensors arrive rapidly, both in real-time streams and in batch mode, and the overall volume is considerable. Ensuring data quality and integrity is also essential, as these data hold significant value for analysis and decision-making. To address these challenges associated with the 4Vs of Big Data, a data lake has been implemented as the core of the batch layer. Unlike a traditional data warehouse with rigid schemata, the data lake stores data in their native format—structured, semi-structured, or unstructured—providing flexibility for integrating new sources and performing various analytical processes. Note that in our approach we use a light integration approach, since we adopt a data lake architecture, where data are stored as they are, without any integration processing tool. In [1] we detail how integration is done since we present a data warehouse approach. Mainly, ETL tools with predefined routines have been implemented, since data sources are known in advance.

To efficiently manage this large volume of data, a Hadoop cluster has been deployed in the cloud, currently composed of three machines. Hadoop is an open-source framework for distributed storage and processing of large-scale data and relies on the HDFS (Hadoop Distributed File System), a distributed file system that splits files into blocks, replicates them across multiple nodes to ensure fault tolerance, and enables parallel data processing across the cluster. The cluster is also scalable, allowing additional machines to be added as needed to increase storage capacity and computational power.

The data lake thus centralizes data from PostgreSQL/PostGIS databases (information on robots and plots), IoT sensors (environmental measurements such as temperature, and soil and air humidity), agricultural robots (operational data collected in real-time via ROS and Kafka), and images for spatial analysis of temperature variations. This approach ensures efficient storage, traceability, and valorization of data, providing a solid foundation for future analyses and predictive applications within the *LambdAgrIoT* ecosystem.

Historical data analysis relies on several technologies within the Hadoop ecosystem, each suited to different types of data. For data originating from robots and PostgreSQL/PostGIS databases, Hive is used. Hive is a query engine based on HiveQL, a SQL-like language that allows querying large datasets stored in HDFS. JSON files are processed using Apache Drill, an interactive query engine that enables working with semi-structured or unstructured data without a predefined schema. ROSBag files, which contain operational robot data, are analyzed using Apache Spark, a distributed, in-memory computing framework designed for fast processing of large-scale datasets.

For visualization and data exploration, Apache Superset, also part of the Hadoop ecosystem, is employed. It connects to Hive to query various tables, build interactive dashboards, and visualize collected data. Superset enables the execution of diverse analytical queries, providing actionable insights into agricultural operations. An example of a Superset dashboard is shown in Figure 5, answering questions such as the following:

1. *Which equipment was used by each robot, by plot, and by year?*
2. *How many times was each piece of equipment used by each robot, by plot, and by year?*
3. *How many robots and pieces of equipment are associated with a specific activity, by plot and year?*

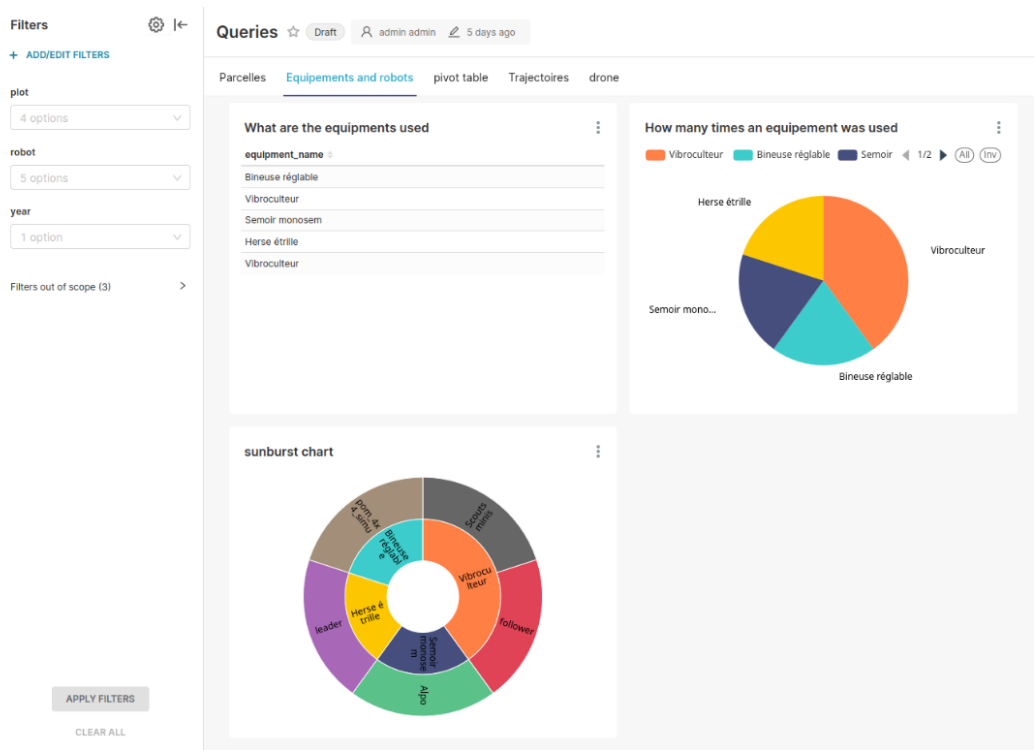

Figure 5. Batch layer: Superset visualisation.

This combination of tools allows for centralized historical data analysis and dynamic visualization, facilitating the monitoring of robots performance and the optimization of agricultural operations over time.

7. Ingestion Layer

This section describes the data ingestion component. Once received by the MQTT broker, the data are redirected to an Apache Kafka broker, a distributed system optimized for ingesting and processing real-time data streams. The streams collected by Kafka are then filtered by type (e.g., soil moisture, air temperature, etc.) before being stored daily in the data lake as separate CSV files, ensuring a structured organization and effective traceability of environmental and robotic data.

To enable centralized collection and analysis, ROS topics are linked to corresponding Kafka topics, through which the data are transmitted, stored, and made available for real-time processing. This mechanism ensures fast and reliable information flow, with a high transmission frequency—typically 10 Hz (ten times per second)—thus providing continuous and precise monitoring of the robots' performance and operational status.

8. Related Work

Several works investigate Human Robot Interaction methods for vehicles used in different application domains. Recently, ref. [15] provides a complete survey of works dedicated to the agricultural context. The authors group existing works according to the agricultural task: target detection, harvesting, and robot navigation. As pointed out by [15], very few works address the navigation problems of robot fleets in the agricultural context. Ref. [16] proposes a system based on Augmented Reality (AR) to monitor a fleet of robots. The system allows visualization of the status and its causes for each robot in real time. Other data from robots are also visualized, such as speed, steering angle, etc. Visualization is integrated with planning software and implemented with AR glasses. The authors do not detail how the priority of alerts when dealing with multiple robots is visualized by their system, nor how the remote control of robots is supported. Ref. [17] proposes an evaluation of two user interfaces for the monitoring of a single spraying robot. The two developed interfaces use multiple views with video from the embedded robot camera. Some basic data are shown, such as battery level and distance for the predefined trajectory. The control of the robot is possible using the keyboard or the mouse. In the same line, ref. [18] compares the usability of remote navigation control of a robot (in two cases: spraying and grasping) with mouse and tangible interfaces. In the more general context of unmanned vehicles several works investigate human-vehicles interaction as surveyed in [19]. Ref. [19] classes existing works according to the type of feedback (visual, auditory, and haptic); all possible information that must be communicated to humans (obstacles, speed, etc.). No details are provided about how alert priorities are handled. Moreover, it focuses solely on one in-vehicle interaction (as in [17] for agricultural robots). Other works study the monitoring of vehicles in different contexts, putting emphasis on geovisualization methods to represent their trajectory data, such as for air traffic flights [20], unmanned vehicles [21,22], which studies the usage of colored icons to convey the urgency levels of alerts. To conclude, to the best of our knowledge, no work proposes a monitoring system for fleets autonomous agricultural robots, as also underlined by the recent survey of [15].

Ref. [8] analyzes two main types of DSMS: distributed engines, which run on multiple nodes and prioritize scalability and high real-time performance, and non-distributed engines, which operate on a single node and focus on lightweight operation.

Among the eight distributed solutions reviewed, each presents trade-offs. GeoFlink excels at real-time spatial streaming but lacks support for batch processing of historical data. Conversely, Apache Sedona is strong in large-scale, distributed batch processing but lacks real-time capabilities. GeoMesa provides storage for massive spatio-temporal data but has high resource requirements and complex deployment. Others, like GeoTrellis, are limited by their focus on raster data, whereas solutions as LocationSpark are now out-

dated. NebulaStream offers high real-time spatial processing but suffers from immaturity and complexity.

In the non-distributed category, Esper is a lightweight, easily deployable solution that offers Complex Event Processing (CEP) and advanced temporal queries, but it critically lacks native support for spatial data. Apache Edgent, while optimized for IoT edge devices, has been retired and lacks advanced features.

Finally, the authors highlight the need for lightweight spatial databases to handle processing at resource-constrained edge devices. Traditional database engines are too heavy for this environment. Viable options include in-memory solutions like DuckDB and H2GIS, which facilitate rapid spatial data processing and are known for their lightweight architectures. Other alternatives for large-scale management include Apache GeoParquet (optimized columnar storage), Apache Iceberg, and Apache IoTDB (tailored for time series). The paper also notes in-process databases such as Apache Derby and RaimaDB as suitable embedded systems for mobile and IoT applications.

9. Conclusions and Future Work

In this paper, we presented a comprehensive smart farming architecture designed to effectively manage the high-volume and diverse data inherent in modern agricultural practices. This architecture, structured into five distinct, interconnected layers—Source, Ingestion, Batch, Speed, and Governance—provides a robust framework for handling both historical and real-time data. The *Source Layer* acts as the crucial entry point, accommodating varied data types, including spatial, image, and streaming numerical data from IoT devices, drones, and robots. The subsequent layers ensure efficient data flow, processing, and management: the *Ingestion Layer*, deployed at the farm level using a fog approach, uses a Python-based batch process for historical data and Kafka for high-throughput real-time stream ingestion; the *Batch Layer*, deployed on the cloud for scalability, leverages a Hadoop cluster, Spark, Hive, and Drill for large-scale historical analysis, with results visualized via Superset; the *Speed Layer* enables low-latency, real-time processing using Geoflink and a dedicated PostGIS database, supported by an ad-hoc geovisualization interface. This layered approach offers a scalable, resilient, and specialized solution, capable of transforming raw farm data into actionable insights for modern smart agriculture.

Our ongoing work concerns the full benchmarking of the proposed solutions. Moreover, we plan to test the use of different in-memory databases such as DuckDB at the edge level [23], and benchmark them. Moreover, we are working on integrating the original data warehouse implementation of the Batch Layer, as proposed in [1], into this new architecture. This integration will allow for OLAP queries over the data stored in the data lake.

Future work includes the extension of spatial operators we have implemented in Esper with trajectory operators in order to improve the analysis capabilities of our supervision and control system. Finally, we also plan to use NATS broker as an alternative to MQTT. This broker (supported by the CNCF) is lightweight, provides some data recovery mechanisms and has an edge-native architecture (it allows to connect NATS instances deployed at far-edge nodes).

Author Contributions: Conceptualization, S.B.; software, M.E.-O.; validation, M.E.-O.; investigation, S.B.; writing—riginal draft preparation, M.E.-O. and N.T.; writing—review and editing, M.E.-O. and S.B.; supervision, N.T.; funding acquisition, S.B. and N.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the CHIST-ERA grant ANR-24-CHR4-0004-0 'GIS4IoRT'.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. André, G.; Bachelet, B.; Battistoni, P.; Belhassena, A.; Bimonte, S.; Cariou, C.; Chabot, F.; Chalhoub, G.; Couvent, A.; Garani, G.; et al. LambdAgrIoT: A new architecture for agricultural autonomous robots' scheduling: From design to experiments. *Clust. Comput.* **2023**, *26*, 2993–3015. [CrossRef]
2. Bellon Maurel, V.; Huyghe, C. Putting agricultural equipment and digital technologies at the cutting edge of agroecology. *Ocl* **2017**, *24*, D307. [CrossRef]
3. Arvanitis, K.G.; Symeonaki, E.G. Agriculture 4.0: The Role of Innovative Smart Technologies Towards Sustainable Farm Management. *Open Agric. J.* **2020**, *14*, 130–135. [CrossRef]
4. Debauche, O.; Mahmoudi, S.; Manneback, P.; Lebeau, F. Cloud and distributed architectures for data management in agriculture 4.0: Review and future trends. *J. King Saud Univ. Comput. Inf. Sci.* **2022**, *34*, 7494–7514. [CrossRef]
5. Bazza, H.; Bimonte, S.; Rizzi, S.; Badir, H. Data management and processing for IoT & robotics in smart farming: A survey. *J. Comput. Lang.* **2025**, *85*, 101355. [CrossRef]
6. Fountas, S.; Carli, G.; Sørensen, C.G.; Tsiropoulos, Z.; Cavalaris, C.; Vatsanidou, A.; Liakos, B.; Canavari, M.; Wiebensohn, J.; Tisserye, B. Farm management information systems: Current situation and future perspectives. *Comput. Electron. Agric.* **2015**, *115*, 40–50. [CrossRef]
7. Ardagna, C.A.; Bellandi, V.; Ceravolo, P.; Damiani, E.; Finazzo, R. A methodology for cross-platform, event-driven Big Data analytics-as-a-service. In Proceedings of the 2019 IEEE International Conference on Big Data (Big Data), Los Angeles, CA, USA, 9–12 December 2019; pp. 3440–3448.
8. Kassir, M.; Bimonte, S.; Wrembel, R.; El-Ouati, M.; Sakr, M. Real-Time Monitoring and Active Control of Autonomous Agricultural Robot Trajectories Using an Edge-Fog Architecture. In *International Conference on Computational Science and Its Applications*; Springer: Berlin/Heidelberg, Germany, 2025; pp. 263–280.
9. MacEachren, A.M.; Gahegan, M.; Pike, W.; Brewer, I.; Cai, G.; Lengerich, E.; Hardisty, F. Geovisualization for Knowledge Construction and Decision Support. *IEEE Comput. Graph. Appl.* **2004**, *24*, 13–17. [CrossRef] [PubMed]
10. Endsley, M.R.; Garland, D.J. *Situation Awareness: Theory, Analysis and Measurement*; CRC Press: Boca Raton, FL, USA, 2000.
11. Parent, C.; Spaccapietra, S.; Zimányi, E. The MurMur project: Modeling and querying multi-representation spatio-temporal databases. *Inf. Syst.* **2006**, *31*, 733–769. [CrossRef]
12. Golab, L.; Özsu, M.T. *Data Stream Management*; Springer Nature: Maharashtra, India, 2022.
13. Kumar, S.A.; Ilango, P. The impact of wireless sensor network in precision agriculture: A review. *Wirel. Pers. Commun.* **2018**, *98*, 685–698. [CrossRef]
14. Dincă, A.M.; Axinte, S.D.; Bacivarov, I.C. In-Memory Versus On-Disk Databases: Best Practices, Use Cases and Architectural Designs. In Proceedings of the 2023 15th International Conference on Electronics, Computers and Artificial Intelligence (ECAI), Bucharest, Romania, 29–30 June 2023; pp. 1–6.
15. Benos, L.; Moysiadis, V.; Kateris, D.; Tagarakis, A.C.; Busato, P.; Pearson, S.; Bochtis, D. Human–robot interaction in agriculture: A systematic review. *Sensors* **2023**, *23*, 6776. [CrossRef] [PubMed]
16. Huuskonen, J.; Oksanen, T. Augmented reality for supervising multirobot system in agricultural field operation. *IFAC-PapersOnLine* **2019**, *52*, 367–372. [CrossRef]
17. Adamides, G.E. Heuristic Evaluation of the User Interface for a Semi-Autonomous Agricultural Robot Sprayer. *AGRIS Online Pap. Econ. Inform.* **2020**, *12*, 3–12. [CrossRef]
18. Mallas, A.; Rigou, M.; Xenos, M. Comparing the Performance of Experts and Farmers when Operating Agricultural Robots. *Hum. Behav. Emerg. Technol.* **2022**, *2022*, 6070285. [CrossRef]
19. Capallera, M.; Angelini, L.; Meteier, Q.; Abou Khaled, O.; Mugellini, E. Human-Vehicle Interaction to Support Driver's Situation Awareness in Automated Vehicles: A Systematic Review. *IEEE Trans. Intell. Veh.* **2022**, *8*, 2551–2567. [CrossRef]
20. Boehm, K.; Roth, V.; Kelley, J. Enhancing Situation Awareness in Real Time Geospatial Visualization. In Proceedings of the Eleventh Americas Conference on Information Systems, AMCIS 2005, Omaha, NE, USA, 11–15 August 2005.
21. Fuchs, C.; Ferreira, S.; Sousa, J.; Gonçalves, G. Adaptive consoles for supervisory control of multiple unmanned aerial vehicles. In *International Conference on Human-Computer Interaction*; Springer: Berlin/Heidelberg, Germany, 2013; pp. 678–687.

22. Friedrich, M.; Vollrath, M. Urgency-Based color coding to support visual search in displays for supervisory control of multiple unmanned aircraft systems. *Displays* **2022**, *74*, 102185. [CrossRef]
23. Hoang, N.N.; Pham, N.H.; Hoang, V.P.; Zimányi, E. MobilityDuck: Mobility Data Management with DuckDB. *arXiv* **2025**, arXiv:2510.07963. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.